A7: Evaluate and Improve AI Capability

Course: Platform Development
Week: 09 – Evaluate and Improve AI Capability

Student Name: Sawitta Thiabsaeng
Student ID: 6731503123
Project / Capability Name: Support Request Classifier
Assignment: A7 – Evaluate and Improve AI Capability

1. Prepare Grounding Context

1.1 AI Capability

The project is a Support Request Classifier that classifies customer support messages into three categories:

- **ACCOUNT_ISSUE:** Problems related to accounts, login, and passwords.
- **TECHNICAL_ISSUE:** Problems related to websites, applications, and technical errors.
- **GENERAL_INQUIRY:** General questions about services and support information.

The system also uses a deterministic fallback mechanism to handle unclear messages, invalid AI responses, model errors, and requests that may involve multiple issues.

1.2 Knowledge Base and Retrieval

The knowledge base is stored in `knowledge_base.json` and contains six records.

ID	Knowledge Topic	Purpose
KB01	Account and login issues	Identify account-related problems
KB02	Password recovery	Support password recovery requests
KB03	Website and server errors	Identify website and server problems

KB04	Application crashes and bugs	Identify technical problems
KB05	Support opening hours	Answer questions about support hours
KB06	General information and services	Handle general inquiries

The retrieval method is implemented in `retrieve_context.py` using simple keyword matching. The system searches for relevant knowledge records based on words in the user's message.

For example, the message “I forgot my password and cannot log in” retrieves KB01, while “The website shows a 500 error” retrieves KB03.

However, a vague message such as “It does not work. Please help” may not retrieve any relevant record. This shows that keyword-based retrieval can be limited when the user provides insufficient information or uses unexpected wording.

1.3 Grounding Context Example

The following is an example of context that could be supplied to the AI model:

> You are a support request classifier. Use the following knowledge base to help interpret the user's message.
>
> - KB01: Account and login issues include problems with logging in and accessing an account.
> - KB02: Password recovery includes forgotten passwords and password-reset problems.
> - KB03: Website and server errors include HTTP errors and website failures.
> - KB04: Application crashes and bugs include applications that crash or fail to load.
> - KB05: Support opening hours contains information about when support is available.
> - KB06: General information and services includes general questions about available services.
>
> Classify the user message into one category: ACCOUNT_ISSUE, TECHNICAL_ISSUE, or GENERAL_INQUIRY.

If the message is ambiguous or lacks information, do not make an unsupported assumption.

This is an example of grounding instructions. Automatic integration between the local retrieval script and the Jev Playground model has not been demonstrated.

1.4 Manual Jev Grounding Experiment

A manual experiment was conducted in Jev Playground to compare the model's decision with and without additional knowledge context. The same customer message, question, and three classification choices were used in both runs.

Test message: “My password reset page is not loading.”

Metric	Without Context	With Context
Selected category	TECHNICAL_ISSUE	TECHNICAL_ISSUE
ACCOUNT_ISSUE probability	27%	36%
TECHNICAL_ISSUE probability	73%	64%
GENERAL_INQUIRY probability	0%	0%
Reported latency	1,071 ms	548 ms

The selected category remained TECHNICAL_ISSUE in both runs. After adding context, the displayed probability for ACCOUNT_ISSUE increased by 9 percentage points, while TECHNICAL_ISSUE decreased by 9 percentage points.

This suggests that the additional context influenced the model's displayed probability distribution in this experiment, although it did not change the final category. Only one message was tested under each condition, so the experiment does not establish that the context improved classification accuracy.

The reported latency was lower in the second run, but two individual measurements are insufficient to conclude that context reduced response time. More repeated tests would be required for a reliable comparison.

The experiment was performed manually in Jev Playground. The local retrieval script has not been demonstrated as automatically providing context to the model.

Screenshots of both runs are saved in the evidence folder:

- `A7_grounding_without_context.png`
- `A7_grounding_with_context.png`

2. Create Evaluation Dataset

The evaluation dataset contains 10 test cases in `eval-cases.json`. The cases cover normal requests, edge cases, fallback situations, and difficult or ambiguous requests.

Case	Type	Input	Expected Decision / Action
E01	Normal	I forgot my password and cannot log in.	ACCOUNT_ISSUE / ROUTE_TO_SUPPORT
E02	Normal	The website shows a 500 Internal Server Error.	TECHNICAL_ISSUE / ROUTE_TO_SUPPORT
E03	Normal	What are your support opening hours?	GENERAL_INQUIRY / ROUTE_TO_SUPPORT
E04	Edge	I cannot log in.	ACCOUNT_ISSUE / ROUTE_TO_SUPPORT
E05	Edge	The app crashes when I open it.	TECHNICAL_ISSUE / ROUTE_TO_SUPPORT

E06	Edge	Can you tell me what services are available?	GENERAL_INQUIRY / ROUTE_TO_SUPPORT
E07	Fallback	It does not work. Please help.	ASK_FOR_MORE_INFORMATION
E08	Fallback	Empty input	ASK_FOR_MORE_INFORMATION
E09	Difficult / Ambiguous	I cannot access my account, and the website keeps showing an error.	SEND_TO_HUMAN
E10	Difficult / Ambiguous	My password reset page is not loading.	SEND_TO_HUMAN

The dataset evaluates expected classifications where applicable and the action taken by the fallback mechanism. For difficult cases, sending the request to a human helps prevent unsupported decisions.

The original 10 cases were retained for before-and-after comparison. Additional targeted tests T09–T11 were conducted separately and are not counted as part of the original 10-case evaluation.

3. Measure Baseline Performance

The baseline evaluation was conducted using the original fallback implementation before the improvement.

The local evaluation script was run using:

```
None
python .\evaluate.py
```

The benchmark was conducted using 10 test cases and 5 repetitions per case, resulting in 50 runs.

3.1 Baseline Results

Metric	Baseline Result
Test cases	10
Total benchmark runs	50
Local action agreement	100%
Average local latency	0.043 ms
Approximate p95 latency	0.063 ms
API cost per task	N/A

The local action agreement was 100%, meaning that all 10 cases produced actions matching the expected actions in the local evaluation dataset.

However, this result does not represent the actual classification accuracy of the AI model. The benchmark measures local retrieval and fallback processing, not the model's prediction quality or network response time.

The API cost per task is reported as N/A because the benchmark did not use a model API or measure token consumption.

4. Improve One Thing

4.1 Selected Improvement

The selected improvement was to expand the fallback rule for password-reset or password-recovery pages that fail.

The rule checks for password-reset or password-recovery page terms together with phrases indicating a failure. The added phrases are:

- `is broken`
- `does not work`
- `doesn't work`

The purpose is to identify requests that may involve both account recovery and technical problems and send them for human review instead of routing them automatically based only on the predicted account category.

4.2 Implementation

The updated implementation is saved in `fallback.py` and `fallback_after.py`. The original version is preserved as `fallback_before.py`.

This makes it possible to compare the original and updated implementations.

The improvement does not allow the system to resolve the issue automatically. Instead, the fallback mechanism sends potentially ambiguous requests for human review.

4.3 Evaluation After Improvement

The updated implementation was evaluated using the same original 10 cases.

Metric	Before	After
Original evaluation cases	10	10
Total benchmark runs	50	50
Local action agreement	100%	100%

Average benchmark latency	0.043 ms	0.050 ms
Approximate p95 latency	0.063 ms	0.079 ms
API cost per task	N/A	N/A

The local action agreement remained at 100%. The latest benchmark showed a small increase in local execution time after the change. These very small timing differences may be affected by measurement noise and should not be interpreted as proof that the overall AI system became slower.

The benchmark measures local retrieval and fallback processing only. It excludes model inference and network latency.

4.4 Targeted Test Results

Before the change, T09 returned ROUTE_TO_SUPPORT, which did not match the intended SEND_TO_HUMAN behavior. After the change, the targeted tests passed.

Test	Input	Expected Action	Result
T09	The password reset page is broken.	SEND_TO_HUMAN	PASS
T10	The password reset page does not work.	SEND_TO_HUMAN	PASS
T11	The password recovery page is broken.	SEND_TO_HUMAN	PASS

All three targeted tests passed using `fallback_after.py`.

The original 10-case evaluation remained at 10/10 action agreement. E10 already triggered human review before the change, so its result did not change.

Therefore, the improvement fixed the newly identified paraphrase behavior and passed three targeted tests, but it did not increase the score on the original 10-case dataset. No test case in the original evaluation showed a decrease in action agreement.

The targeted tests are additional to the original dataset and are not included in the original 10-case agreement score.

5. Engineering Reflection

5.1 What I Learned

This assignment helped me understand that implementing an AI capability is not enough. The system must also be evaluated using a consistent dataset.

Testing only a few successful examples is insufficient because real support messages may be incomplete, ambiguous, or contain multiple issues.

I also learned that deterministic fallback rules can help manage uncertain inputs. For example, a system can ask the user for more information or send a request to a human instead of making an unsupported decision.

The targeted tests also demonstrated the importance of testing different ways users may describe the same problem. Adding a rule does not automatically improve the original evaluation score, so the change must be tested against both the original dataset and new targeted cases.

5.2 Limitations

There are several limitations in the current project:

1. The knowledge retrieval method relies on keyword matching and may fail to identify vague messages or paraphrases.
2. The evaluation measures local action agreement rather than actual AI model accuracy.
3. The grounding context has not been demonstrated as automatically integrated with the Jev Playground model.
4. The benchmark excludes model inference time, network latency, and API costs.
5. The original evaluation dataset contains only 10 cases and may not represent all real-world support requests.
6. The grounding experiment used one message per condition, which is insufficient to establish a reliable accuracy or latency improvement.

5.3 Next Steps Before Production

Before deploying the system in production, I would:

- Expand the evaluation dataset with more real-world and ambiguous examples.
- Test whether retrieved knowledge improves actual model predictions across multiple messages.
- Measure classification accuracy, precision, recall, and confusion between categories using human-labelled ground truth.
- Evaluate fallback behavior on incomplete, invalid, and conflicting model outputs.
- Test the system with human reviewers and collect feedback.
- Measure full end-to-end latency and API cost if a model API is integrated.

6. Conclusion

The A7 evaluation provided a reproducible comparison between the original and updated fallback implementations.

Both versions achieved 100% local action agreement on the original 10-case dataset. The selected improvement corrected the targeted password-recovery paraphrase behavior, and T09–T11 passed after the change.

The improvement did not change the results of the original 10-case evaluation because the existing fallback logic already handled E10. The grounding experiment changed the displayed probability distribution but did not change the selected category. The experiment is too small to demonstrate improved classification accuracy.

The current implementation should therefore be treated as a prototype rather than a production-ready support classification system. Further testing with a larger dataset and actual model integration is needed before production deployment.